

Universidade Estadual de Santa Cruz (UESC)

Etapa 1 — Benchmark de Geração do Conjunto de Mandelbrot
(Escape-Time): Implementação sequencial e paralela em memória
compartilhada com OpenMP.

Relatório acadêmico — DEC107 — Processamento Paralelo

Autores: André Nascimento Pardino; Cauã Clemente Lima

Disciplina: DEC107 — Processamento Paralelo

Data: 22/09/2026

1. Introdução

Este projeto tem como objetivo analisar como o paralelismo, a partir do uso de OpenMP, pode melhorar o desempenho do cálculo do conjunto de Mandelbrot e, além disso, observar quais as melhores abordagens para o mesmo.

O conjunto de Mandelbrot é o conjunto dos números complexos c para os quais a sequência definida por $z_0 = 0$ e $z_{n+1} = z_n^2 + c$ permanece limitada. Na prática, para cada ponto c do plano complexo, itera-se essa recorrência e conta-se quantas iterações são necessárias até que $|z|$ ultrapasse um raio de escape (tipicamente 2). Se o ponto não escapar dentro de um número máximo de iterações, considera-se que ele pertence ao conjunto. O número de iterações até o escape (escape time) é o valor utilizado para colorir cada pixel da imagem do fractal.

O problema se baseia no fato de que o custo computacional de cada pixel está relacionado com o seu escape (e portanto sua quantidade de iterações) e, como a princípio a quantidade de iterações por pixel e de iterações totais por cada linha da imagem são significativamente distintas umas das outras, as linhas terão custos computacionais variáveis. Sendo assim, mesmo que o problema em si seja paralelizável, compartilhando memória e dividindo a tarefa em linhas para threads diferentes trabalharem ao mesmo tempo, ele pode apresentar um alto desbalanceamento de carga por conta da imprevisibilidade da quantidade de iterações.

Portanto, este projeto visa, além de construir um código com memória compartilhada para resolução de um problema, também abordar estratégias de distribuição de trabalho que

diminuam o desbalanceamento de carga desta divisão de tarefas, analisando como o número de threads, chunks e políticas de escalonamento como *static*, *dynamic* e *guided* influenciam no desempenho da aplicação.

2. Metodologia

A implementação sequencial de referência, utilizada para basear a implementação paralela em OpenMP e posteriormente utilizada para análise de corretude e *speedup*, utiliza de dois argumentos: o tipo de teste, cujas opções são “padrão” (referente à visão completa compreendida em $Re \in [-2,0, 1,0]$, $Im \in [-1,5, 1,5]$) e “cavalos” (referente à região de *zoom* no “vale dos cavalos-marinhos”, centrada em $(-0,743643887, 0,131825904)$ com largura $3,0 \times 10^{-3}$ no eixo real); e a resolução da imagem, que será utilizada para largura e altura. O padrão de execução para os argumentos são padrão e 4096 respectivamente. Por se tratar de uma região de alto fator de balanceamento de carga, a região dos cavalos marinhos é utilizada para os casos de teste de estresse, e o máximo de iterações é aumentado de 1000 para 5000.

O cálculo de cada pixel é feito na função *calcular_mandelbrot()*, que salva todos os pixels no array *count* alocado dinamicamente. O tempo de cálculo é armazenado e exibido na tela, assim como o tempo de salvamento dos arquivos “.bin” para corretude e “.ppm” para visualização e o tempo total.

A implementação paralela com OpenMP utiliza dos mesmos argumentos de entrada, mas com a possibilidade de configurar variáveis do OpenMP, sendo estas:

- OMP_NUM_THREADS, que define quantas threads estarão disponíveis para a execução paralela, sendo relevante para avaliar o comportamento do código com diferentes quantidades de threads para análise de escalabilidade;
- OMP_SCHEDULE, que define qual política de escalonamento será utilizada (*static*, *dynamic*, *guided*), além de definir também o tamanho dos *chunks*, que determina quantas iterações consecutivas do laço são agrupadas em cada bloco de trabalho atribuído às threads.

A implementação paralela utiliza uma região *parallel* para criar a equipe de threads e uma diretiva *for* para distribuir as iterações do cálculo entre essas threads. A diretiva *parallel* permite a execução concorrente das threads, enquanto *omp for* realiza a divisão das iterações do laço entre os membros da equipe.

Para o escalonamento do trabalho, foi utilizada a cláusula *schedule(runtime)*, que permite definir a política de distribuição das iterações em tempo de execução por meio da variável de ambiente OMP_SCHEDULE. Dessa forma, podem ser avaliadas diferentes políticas de escalonamento, como *static*, *dynamic* e *guided*, além de diferentes tamanhos de *chunk*, sem necessidade de recompilar o programa. A cláusula *nowait* é utilizada para remover a barreira implícita ao final do *omp for*, permitindo que as threads que terminarem suas iterações continuem a execução sem precisar aguardar as demais.

O hardware de execução dos testes da aplicação conta com apenas 8 threads disponíveis para paralelizar o trabalho, limitando a escala dos testes.

Para medição do tempo, a versão paralela armazena o tempo de início e fim do cálculo total e também armazena o tempo de cálculo de cada thread para cálculo do fator de balanceamento.

Além disso, o código em linguagem python "*benchmark_omp.py*" é utilizado para automatizar os testes, utilizando de vários parâmetros de cada tipo por vez. Por exemplo, pode-se automatizar um teste de quantidade 1, 2, 4 e 8 threads, cada um desses testes levar em conta chunks de tamanho 1, 2, 4, 8 e 16, repetindo 5 vezes cada caso, sendo $4 \times 5 \times 5$ totalizando 100 casos de teste. Além disso, para cada combinação de tipo de teste e tamanho de resolução desejada, é também executada uma versão de referência feita pelo código serial, de forma que possa ser analisada a corretude e o speedup de cada um desses casos de teste.

O código armazena em um csv as informações relevantes sobre cada caso de teste, como tempo máximo, tempo mínimo, tempo total, fator de balanceamento, speedup, etc. Este csv será utilizado para montar os gráficos a partir do código também em linguagem python "*analisar_resultados.py*" de acordo com a finalidade do teste: o gráfico de *strong scaling*, testes mantendo a resolução base de 4096x4096 e variando o número de threads, leva em conta o tempo, speedup e eficiência por número de threads; o gráfico de *weak scaling*, escalando a resolução e, proporcionalmente, aumentando a quantidade de threads disponíveis, leva em conta o tempo por resolução, anotando também as threads usadas; e o gráfico de comparação de schedules e chunks, variando o tamanho das chunks e políticas de escalonamento, leva em conta o tempo, fator de balanceamento, e relação do fator de balanceamento por tempo de *schedule* e *chunk*.

Para o teste de strong scaling, foram utilizados como argumentos o tipo de teste padrão, a resolução 4096, a política de escalonamento *static*, repetindo 5 vezes, com quantidade de threads 1, 2, 4 e 8 com chunks de tamanho 1.

Para o teste de comparação de schedules e chunks, foram utilizados como argumentos o tipo de teste na região dos cavalos marinhos (que tem alto fator de desbalanceamento e por isso é relevante para este tipo de teste), a resolução 4096, a política de escalonamento *static*, *dynamic* e *guided*, repetindo 5 vezes, com quantidade de threads 8 com chunks de tamanho 1, 16, 64, 256.

Para o teste de weak scaling, foram utilizados como argumentos o tipo de teste padrão, a resolução 4096 e 8192, a política de escalonamento *static*, repetindo 5 vezes, com quantidade de threads 2 e 8 com chunks de tamanho 1.

3. Resultados e discussão

Os arquivos .csv's gerados nos casos de teste descritos anteriormente mostram que a corretude em relação ao código serial está adequada, tendo diferença de 0 pixels em todos os 3 testes:

caso	resolucao	max_iter	threads_utilizadas	schedule_utilizado	chunk_utilizado	n_repeticoes	pixels_diferentes_max	percentual_erro_max	diferenca_maxima_max	corretude_ok
padrao	4096	1000	1	static	1	5	0	0.0000000000	0	SIM
padrao	4096	1000	2	static	1	5	0	0.0000000000	0	SIM
padrao	4096	1000	4	static	1	5	0	0.0000000000	0	SIM
padrao	4096	1000	8	static	1	5	0	0.0000000000	0	SIM

Corretude do caso de teste strong scaling

caso	resolucao	max_iter	threads_utilizadas	schedule_utilizado	chunk_utilizado	n_repeticoes	pixels_diferentes_max	percentual_erro_max	diferenca_maxima_max	corretude_ok
cavalos	4096	5000	8	dynamic	1	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	dynamic	16	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	dynamic	64	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	dynamic	256	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	guided	1	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	guided	16	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	guided	64	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	guided	256	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	static	1	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	static	16	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	static	64	5	0	0.0000000000	0	SIM
cavalos	4096	5000	8	static	256	5	0	0.0000000000	0	SIM

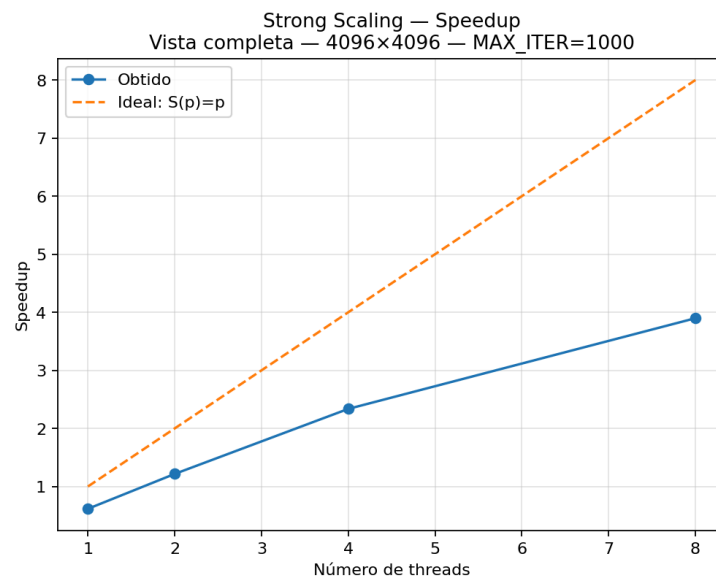
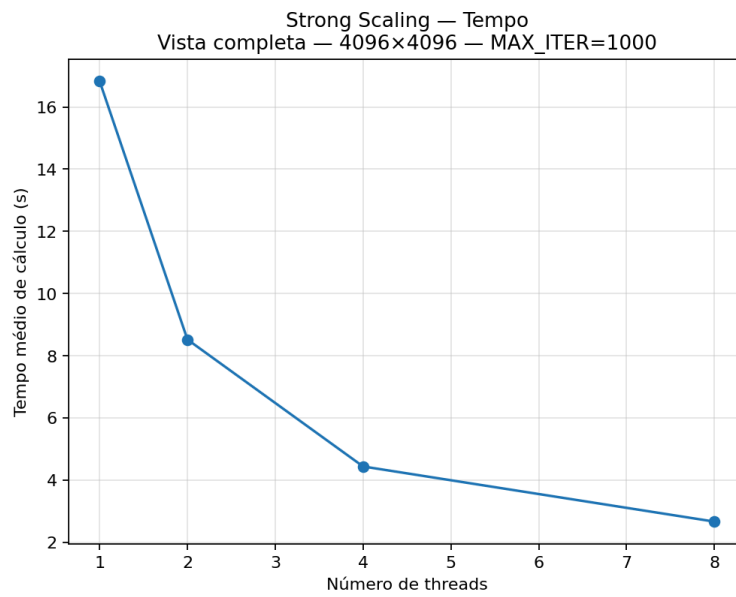
Corretude do caso de teste schedules_chunks

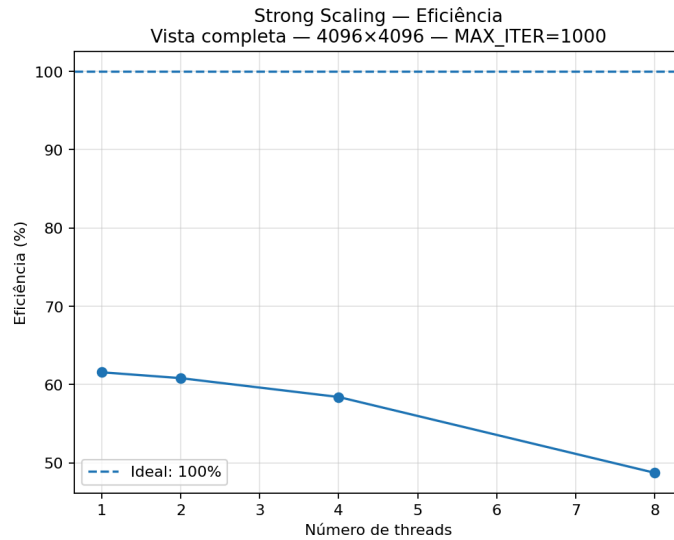
caso	resolucao	max_iter	threads_utilizadas	schedule_utilizado	chunk_utilizado	n_repeticoes	pixels_diferentes_max	percentual_erro_max	diferenca_maxima_max	corretude_ok
padrao	4096	1000	2	static	1	5	0	0.0000000000	0	SIM
padrao	4096	1000	8	static	1	5	0	0.0000000000	0	SIM
padrao	8192	1000	2	static	1	5	0	0.0000000000	0	SIM
padrao	8192	1000	8	static	1	5	0	0.0000000000	0	SIM

Corretude do caso de teste weak scaling

Comprovado a corretude, passamos para a análise de desempenho nos casos de teste em questão.

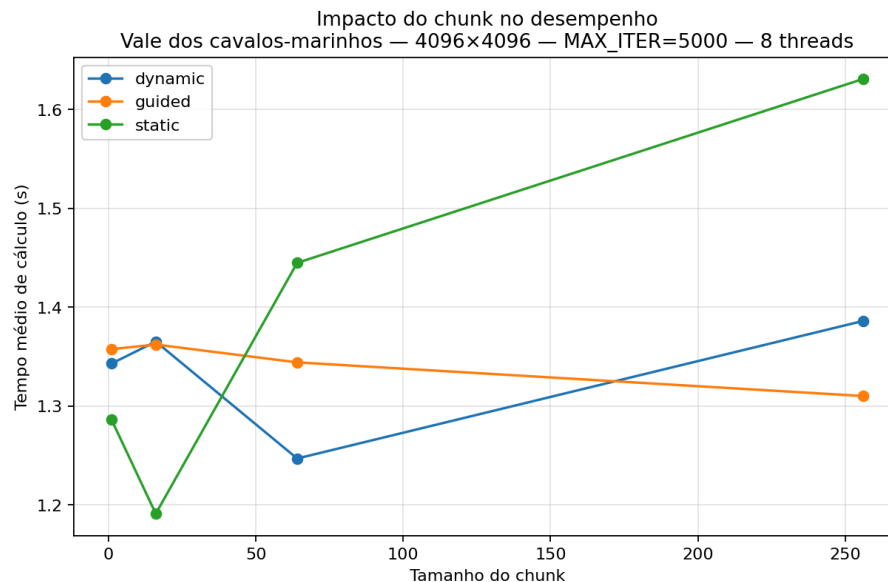
Strong scaling:

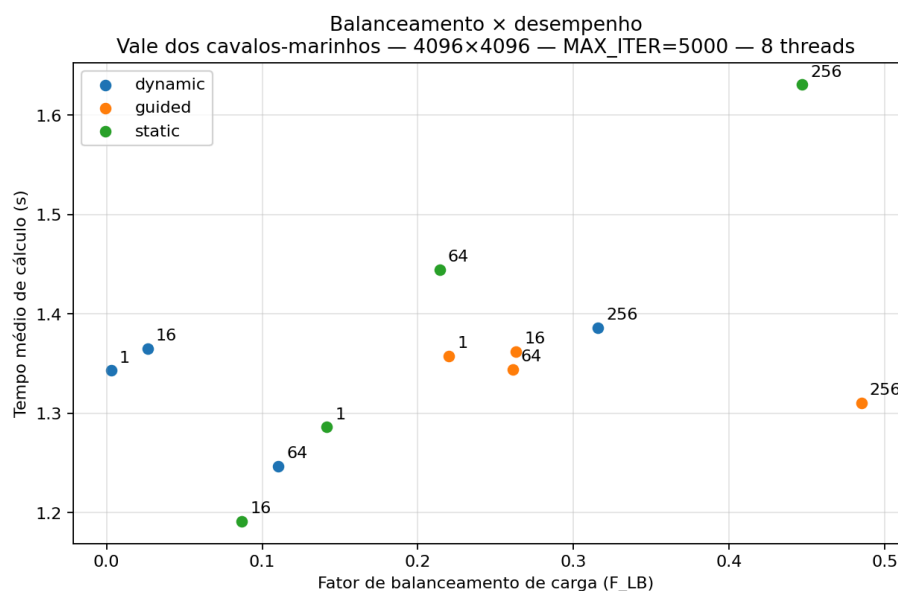
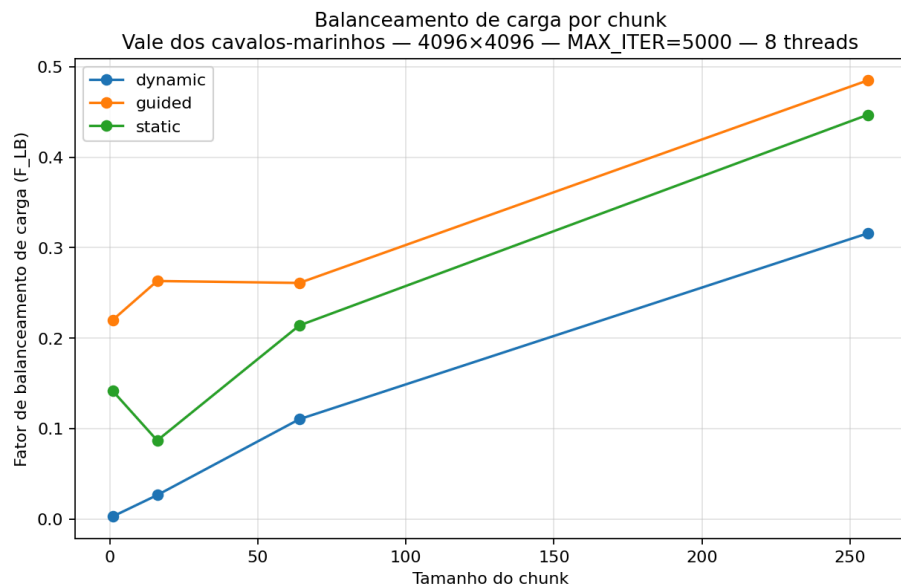




Podemos perceber a partir dos gráficos gerados a partir dos .csv's resultantes que, conforme a adição de threads disponíveis para divisão do trabalho, o tempo de cálculo diminui e o speedup em relação ao serial, como esperado. Mas vale notar que, apesar da melhora, cada adição consecutiva de threads diminui a melhora em relação à adição anterior, relacionado à logica da Lei de Amdahl.

Comparação de schedules e chunks:

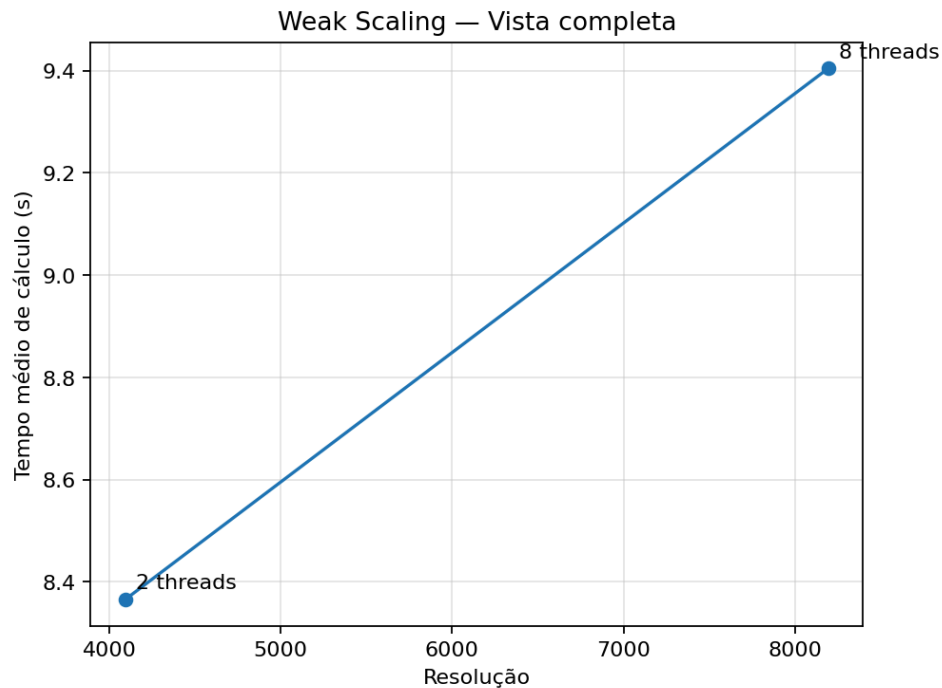




Pode-se notar que, conforme o aumento da quantidade de chunks, o fator de balanceamento aumenta, devido à menor variedade entre possível entre um número menor de chunks. Nota-se também que as políticas de escalonamento static (que divide as iterações entre as threads previamente) e dynamic (que distribui os blocos de trabalho conforme as threads terminam suas tarefas) acabam aumentando o tempo de cálculo com tamanhos maiores de chunks, enquanto que a guided (que também distribui o trabalho dinamicamente, mas começa com blocos maiores e reduz progressivamente seu tamanho) diminui seu tempo, ainda que não tão significativamente.

Percebe-se que a política de escalonamento que apresentou melhor desempenho neste contexto de alto desbalanceamento foi a static com chunks de tamanho 16.

Weak scaling



Apesar da variação de tempo médio entre os casos com 2 e 8 threads, percebe-se que os dois testes, mesmo considerando o salto de complexidade entre 4096x4096 e 8192x8192, manteve um tempo semelhante

O gargalo da implementação, ao menos do código python de benchmark para automatizar os testes, é a necessidade de sempre fazer uma versão serial de referência para cada combinação de tipo de teste por resolução, o que pode aumentar o tempo significativamente, considerando que este leva em torno de 3 ou 4 vezes mais tempo que os testes paralelos em sua maioria. Uma sugestão de melhoria futura seria calcular o tempo da versão serial de referência apenas para cada tipo de teste e a partir de estimativas calcular o tempo entre uma resolução e outra.

5. Conclusão

A etapa 1 do projeto permitiu o aprofundamento dos conhecimentos relacionados ao paralelismo, bem como a habituação à tecnologia de OpenMP. Dentre os conceitos que foram reforçados durante o desenvolvimento, destacam-se: o conceito de speedup e sua relação com a quantidade de threads, como visto no strong scaling; a escalabilidade do paralelismo relacionada à quantidade de threads proporcional ao aumento da complexidade do problema, como visto no teste de weak scaling; fator de balanceamento como um conceito relevante e presente durante todos os testes, mas especialmente no teste de estresse na região dos cavalos marinhos; e o estudo das políticas de escalonamento static, dynamic e guided, tanto de seu funcionamento quanto de sua eficácia no contexto destes testes.

O desempenho do projeto, considerando esta primeira etapa dentro de 3, foi satisfatório, obtendo speedups próximos de 4 nos melhores casos.

6. Referências

- Processamento paralelo: Aula 03 - Caracterização de desempenho em programas paralelos;
- Processamento paralelo: Aula 04 - Arquiteturas Paralelas. Introdução a OpenMP
- Processamento paralelo: Aula 06 - Diretivas OpenMP
- Processamento paralelo: Aula 07 - Diretivas OpenMP
- Processamento paralelo: Aula 08 - Diretivas OpenMP
- https://pt.wikipedia.org/wiki/Lei_de_Amdahl

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, André Nascimento Pardino, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado “parallel-processing-mandelbrot”, submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: [Listar ferramentas, ex: ChatGPT 4.0, GitHub Copilot]. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Auxílio na depuração (debugging) de blocos específicos do código da implementação paralela
- Geração de scripts base para automação de testes de desempenho (benchmark_omp.py) e geração dos gráficos;

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 22/09/2026

André Nascimento Pardino

Matrícula: 202410507

DECLARAÇÃO DE TRANSPARÊNCIA E USO DE INTELIGÊNCIA ARTIFICIAL

Eu, Cauã Clemente Lima, regularmente matriculado(a) no curso de Ciência da Computação, declaro para os devidos fins que, na elaboração do trabalho/projeto intitulado “parallel-processing-mandelbrot”, submetido como requisito para a disciplina/atividade DEC107 — Processamento Paralelo, adotei as seguintes práticas em relação ao uso de ferramentas de Inteligência Artificial (IA) generativa:

(Assinale a opção aplicável)

() Não utilizei ferramentas de Inteligência Artificial na concepção, desenvolvimento, programação ou redação deste trabalho.

(X) Utilizei ferramentas de Inteligência Artificial (ex: ChatGPT, GitHub Copilot, Claude, etc.) como recursos de apoio.

Detalhamento do Uso (se aplicável): As ferramentas utilizadas foram: [Listar ferramentas, ex: ChatGPT 4.0, GitHub Copilot]. O uso se deu exclusivamente nas seguintes etapas do projeto:

- Revisão gramatical no trecho do relatório referente à políticas de escalonamento;
- Geração de scripts para adição da lógica de speedup e corretude no benchmark_omp.py
- Auxílio para compreensão do problema e direcionamento;

Termo de Responsabilidade: Declaro estar ciente de que ferramentas de IA não são isentas de erros, vieses ou alucinações. Sendo assim, atesto que toda a responsabilidade pela autoria, integridade, originalidade e exatidão do conteúdo submetido é exclusivamente minha. Confirmando que todo o código-fonte, dados de testes e textos gerados ou modificados com o auxílio de IA foram rigorosamente revisados, testados e validados por mim, garantindo que os resultados refletem meu próprio entendimento e esforço intelectual, sem incorrer em plágio intelectual ou acadêmico.

Ilhéus - BA, 22/09/2026

Cauã Clemente Lima

Matrícula: 202410120